# Knowledge Representation



## Artificial Intelligence

*School of Computing*
*Universiti Teknologi Malaysia*

# Outline

We will learn these topics:

1. AI-based knowledge representation
2. Propositional Logic
3. Predicate Logic



# 2.1 AI-based knowledge representation

# Representation

- AI agents deal with knowledge (data)
  - Facts (believe & observe knowledge)
  - Procedures (how to knowledge)
  - Meaning (relate & define knowledge)
- Right representation is crucial
  - Early realisation in AI
  - Wrong choice can lead to project failure
  - Active research area

# Choosing a Representation



- For certain problem solving techniques
  - 'Best' representation already known
  - Often a requirement of the technique
  - Or a requirement of the programming language (e.g. Prolog)
- Examples
  - First order theorem proving… first order logic
  - Inductive logic programming… logic programs
  - Neural networks learning… neural networks
- Some general representation schemes
  - Suitable for many different (and new) AI applications

# Some General Representations



1. Logical Representations
2. Production Rules
3. Semantic Networks
   - Conceptual graphs, frames

# Syntax and Semantics

- Syntax
  - Rules for constructing legal sentences in the logic
  - Which symbols we can use (English: letters, punctuation)
  - How we are allowed to combine symbols
- Semantics
  - How we interpret (read) sentences in the logic
  - Assigns a meaning to each sentence
- Example: “All lecturers are seven foot tall”
  - A valid sentence (syntax)
  - And we can understand the meaning (semantics)
  - This sentence happens to be false (there is a counterexample)

# Activity 2.1



**Activity**

1. Watch the video below.
2. Answer YES/NO for sentences given.
3. One student is called to discuss the answer.

https://www.youtube.com/watch?v=zdoIfEKw3eU

**Question**

1. My book is crying.
   Syntax : .....
   Semantic : .....

2. UTM in Johore.
   Syntax : .....
   Semantic : .....

# Activity 2.1 - Answer



**Explanation**

1. Syntax is usually associated with the rules or grammar
2. Semantic is concerned with its meaning.

**Question**

1. My book is crying.
   Syntax : **YES**
   Semantic : **NO**

2. UTM in Johore.
   Syntax : **NO**
   Semantic : **YES**

# But.. what is knowledge?



- Rules?
  - Production systems
- Classification?
  - Semantic networks
- Structure?
  - Frames
- Idealised and formalised knowledge?
  - Logic

# Why need representation?

-capture important features of a problem

-make information accessible to problem solving procedure

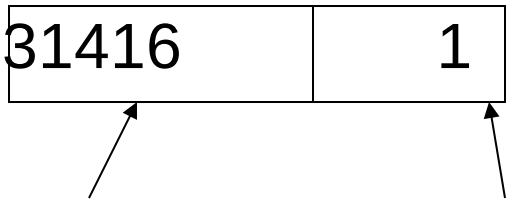
-abstraction

e.g.

The real number: $\pi$

Decimal : 3.1415927….

Floating point :




Computer memory: 11100010

# AI as representation

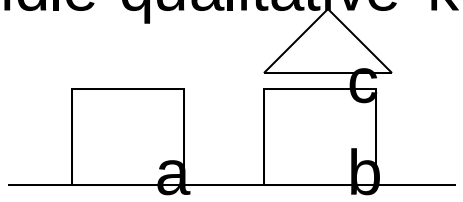
A representational scheme should:

1. Be adequate to express all of the necessary information
2. Support efficient execution of the resulting code
3. Provide a natural scheme for expressing the required knowledge

# AI as representation

To support all these needs, AI representation language must:

1. Handle qualitative knowledge-(reasoning)




clear(c),clear(a), ontable(a), ontable(b)

on(c,b), cube(a), cube(b), pyramid(c)

Predicates : clear, ontable, on, cube, pyramid

# AI as representation



2. Allow new knowledge to be inferred from set of facts and rules

$$\forall X \neg \exists Y \text{ on}(X,Y) \rightarrow \text{clear}(X)$$

-for all X, X is clear if there does not exist a Y such that Y is on X-

3. Allow representation of general principles as well as specific solutions

4. Capture complex semantic meaning

   hassize(bluebird, small), hascovering(bird,feathers), hascolor(bluebird,blue), hasproperty(bird,flies)

5. Allow for meta-level reasoning

   "knowing about what you know" – meta-knowledge

# AI as representation

Knowledge representation languages:

Predicate calculus, semantic networks, frames, objects, rules,..etc.



# 2.2 Propositional Logic

# What is a Logic?

- A language with concrete rules
  - No ambiguity in representation (may be other errors!)
  - Allows unambiguous communication and processing
  - Very unlike natural languages e.g. English
- Many ways to translate between languages
  - A statement can be represented in different logics
  - And perhaps differently in same logic
- **Expressiveness** of a logic
  - How much can we say in this language?
- Not to be confused with logical reasoning
  - Logics are languages, reasoning is a process (may **use** logic)

# Ask google ..what is propositional logic?



- **propositional logic** logic
  (or "propositional calculus") A system of symbolic logic using symbols to stand for whole propositions and logical connectives. Propositional logic only considers whether a proposition is true or false. In contrast to predicate logic, it does not consider the internal structure of propositions. (2002-05-21)

# Basic Logic Operations



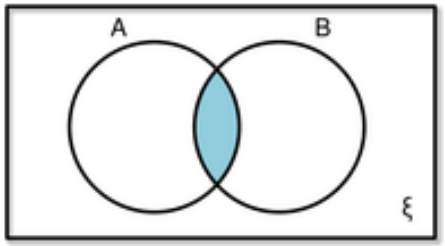
- The NOT operation: ¬
- The AND operation: ∧
- The OR operation: ∨
- The “implication” operation: →
- The “equivalence” operation: ↔ or ≡

# Activity 2.2

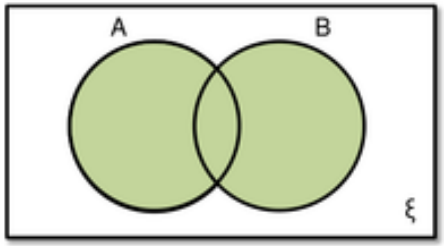


**Activity**

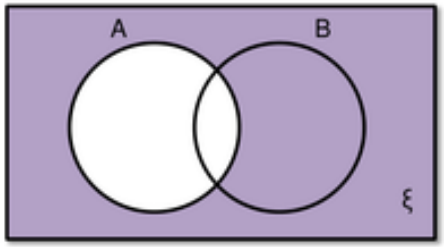
Form a group of two members. Discuss the right logic operation that is suitable for Venn diagram below.




Operation : .....




Operation : .....

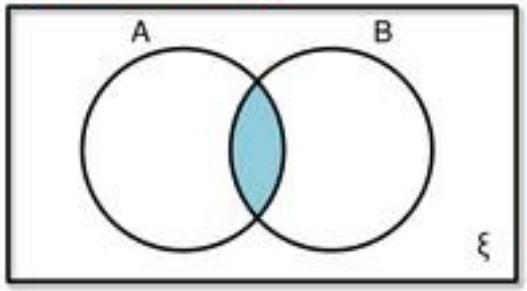



Operation : .....

# Activity 2.2 - Answer

## Venn Diagrams

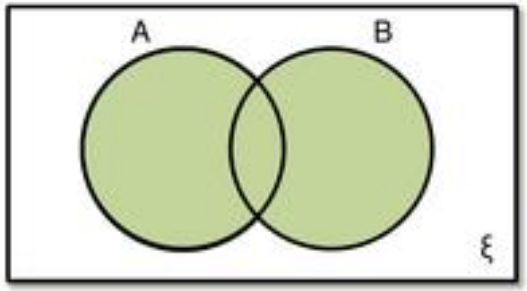
Definitions:




The **intersection** is where two sets overlap.

$A \cap B$

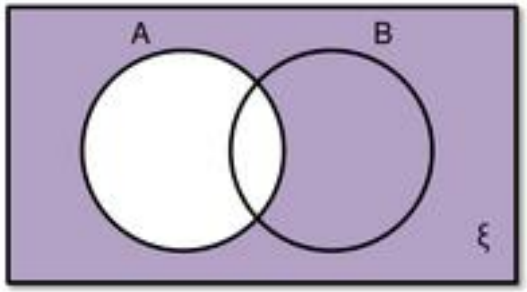
This means **A and B**.




If you put two sets together, you get the **Union**.

$A \cup B$

This means **A or B**. Think marriage, they become 1!




The **complement of A** is the region that is not A.

$A'$

This means **not A**. Or outside of A

# Logic in General (1)

- *Logics* are formal languages for representing information such that conclusions can be drawn
- When people talk about logic they often mean *propositional* or *first-order predicate* logic.

# Logic in General (2)

- A *logic* usually has a well defined syntax, semantics and proof theory.
- The *syntax* of a logic defines the syntactically acceptable objects of the logic, or *well-formed formulae*.
- The *semantics* of a logic associate each formula with a *meaning*.
- The *proof theory* is concerned with manipulating formulae according to certain rules.

# Propositions???

- Pak Lah is Prime Minister
- What's the time?
- 2 + 3 = 5
- ‘Phone’ has five letters.
- 2 + 3 = 6
- Oh dear!
- ‘Work’ has five letters.
- the reactor is on
- *All* elephants have 4 legs
- I like AI class

# Propositional Logic

- The syntax of propositional logic is constructed from *propositions* and *connectives*.
- A **proposition** is a statement that is either true or false but not *both*

# Propositional Logic



- Syntax
  - Propositions, e.g. “it is wet”
  - Connectives: and, or, not, implies, iff (equivalent)
  - Brackets, T (true) and F (false)
- Semantics (C $\wedge$ $\vee$ $\neg$ $\rightarrow$ $\leftrightarrow$
  - Define how connectives affect truth
    - “P and Q” is true if and only if P is true and Q is true
  - Use **truth tables** to work out the truth of statements

# How to determine Propositions??



- It is possible to determine whether any given statement is a proposition by prefixing it with

  *It is true that . . .*

  and seeing whether the result makes grammatical sense.
- Propositions are often abbreviated using *propositional variables* eg $p, q, r$.
- Thus we must associate the propositional variable with its meaning i.e.

  Let $p$ be *Pak Lah is Prime Minister*.

# Connectives



- Propositions may be combined with other propositions to form **compound propositions**. These in turn may be combined into further propositions.
- The connectives that may be used are

| | | |
|---|---|---|
| $\wedge$ | and | conjunction (& or .) |
| $\vee$ | or | disjunction (\| or +) |
| $\neg$ | not | negation (~) |
| $\Rightarrow$ | if . . .then | implication ( $\rightarrow$ ) |
| $\leftrightarrow$ | if and only if | equivalence ( $\Leftrightarrow$ ) |

- Some books use different notations. Some of these are given in parentheses.

# Propositional Logic: Syntax



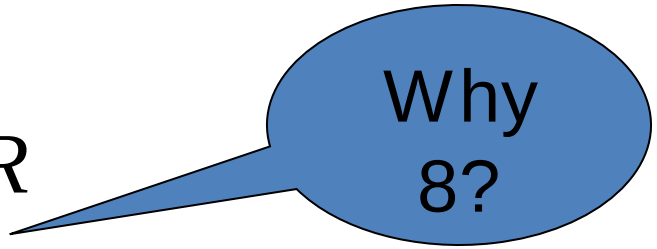
- The proposition symbols $P1$, $P2$ etc are sentences
- If $S$ is a sentence, $\neg$ s a sentence (*negation*)
- If $S_1$ and $S_2$ are sentences, $S_1$ $\wedge$ is a sentence (*conjunction*)
- If $S_1$ and $S_2$ are sentences, $S_1$ $\vee$ is a sentence (*disjunction*)
- If $S_1$ and $S_2$ are sentences, $S_1$ $\Rightarrow$ is a sentence (*implication*)
- If $S_1$ and $S_2$ are sentences, $S_1$ $\Leftrightarrow$ s a sentence (*equivalence*)

# Propositional Logic: Semantic

Each model species true/false for each proposition symbol

E.g. $P$ $Q$ $R$

$true$ $true$ $false$




(With these symbols, 8 possible models, can be enumerated automatically.)

Rules for evaluating truth with respect to a model *m* are determined by *truth tables*

# Propositional Logic: Semantic



- Why 8 models?

| $P$ | $Q$ | $R$ |
|---|---|---|
| *true* | *true* | *false* |

| A | B | C |
|---|---|---|
| False | False | False |
| False | False | True |
| False | True | False |
| False | True | True |
| True | False | False |
| True | False | True |
| True | True | False |
| True | True | True |

**How many variables? 3 variables P, Q and R**
**How many models? $2^3$ = 8 models**

# $\wedge$ And (also called *conjunction*)

- The *conjunction* '$p$ *AND* $q$', written $p \wedge q$, of two propositions is true when both $p$ and $q$ are true, false otherwise.
- We can summarise the operation of $\wedge$ by a Truth table. Rows in the table give all possible setting of the propositions to true (T) or false (F).

| p | q | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

| $p$ | $q$ | $p \wedge q$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $F$ |
| $F$ | $F$ | $F$ |

# Natural Language Meaning (1)



| | |
|---|---|
| $p$ | Its Monday. |
| $q$ | Its raining. |
| $p \wedge q$ | Its Monday and its raining. |
| | Its Monday but its raining. |
| | Its Monday. Its raining. |

# Natural Language Meaning (2)



Caution: semantics?

| | |
|---|---|
| $p$ | I took a shower |
| $q$ | I woke up |
| $p \wedge q$ | "I took a shower and I woke up" |
| $q \wedge p$ | "I woke up and I took a shower". |

Logically the same! *WE* may see difference.

The word *both* is often useful eg. *I both took a shower and I woke up*.

# $\vee$ Or

Also called *disjunction.*

The disjunction '$p$ $OR$ $q$', written $p$ $\vee$, of two propositions is true when $p$ or $q$ (or both) are true, false otherwise.

Sometimes called *inclusive or*.

| p | q | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

| $p$ | $q$ | $p \vee q$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $T$ |
| $F$ | $T$ | $T$ |
| $F$ | $F$ | $F$ |

# Natural Language Meaning



| | |
|---|---|
| $p$ | It's Monday. |
| $q$ | It's raining. |
| $p \vee q$ | It's Monday or it is raining. |

The word *either* is often useful eg. *either it's Monday or it is raining*.

It *also* includes the case of rain on a Monday!

# ¬ Not

Also known as *negation*

The negation 'NOT $p$' of a proposition (or $p$, ¬ is true when p is false and is false otherwise. $p$ may be read that it is false that $p$.

| p | |
|---|---|
| | |
| | |

| $p$ | $\neg p$ |
|---|---|
| $T$ | $F$ |
| $F$ | $T$ |

Negation is a *unary* connective. It only takes one argument.

Conjunction and disjunction were both *binary* connectives.

# Natural Language Meaning



| | |
|---|---|
| $p$ | Logic is easy. |
| $\neg p$ | It is false that logic is easy. |
| | It is not the case that logic is easy. |
| | Logic is not easy. |

# $\Rightarrow$ If . . . then . .



Also known as **implication**

The implication '$p$ IMPLIES $q$', written $p \Rightarrow q$, of two propositions is true when either $p$ is false or $q$ is true, and false otherwise. An if…then statement will be false when $p$ is true and $q$ is false, and will be true for all other cases

| p | q | |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

| $p$ | $q$ | $p \Rightarrow q$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $T$ |
| $F$ | $F$ | $T$ |



Premier Digital Tech University™

# Natural Language Meaning



| | |
|---|---|
| $p$ | I study hard. |
| $q$ | I get rich. |
| $p \Rightarrow q$ | If I study hard then I get rich. |
| | Whenever I study hard, I get rich. |
| | That I study hard implies I get rich. |
| | I get rich, if I study hard. |

# More About Implication



$p \Rightarrow q$ is true in the following situations

- I study hard and I get rich; or
- I don't study hard and I get rich; or
- I don't study hard and I don't get rich.

The last two situations, i.e. when $p$ is false (I don't study hard) we can't say whether I will get rich or not. However if I've studied hard but failed to become rich then the proposition is clearly false.

# Contrapositive

| | |
|---|---|
| $p \Rightarrow q$ | If I study hard then I get rich. |
| $q \Rightarrow p$ | If I get rich then I study hard. |
| | (the **converse.**) |
| $\neg q \Rightarrow \neg p$ | If I don't get rich then I don't study hard. |
| | (the **contrapositive.**) |

# $\Leftrightarrow$ Biconditional

- Also known as iff (if and only if) or the biconditional.
- The biconditional, written as $p \Leftrightarrow q$, of two propositions is true when both $p$ and $q$ are true or when both $p$ and $q$ are false, and false otherwise.

| $p$ | $q$ | $p \Leftrightarrow q$ |
|---|---|---|
| $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ |
| $F$ | $T$ | $F$ |
| $F$ | $F$ | $T$ |

# Biconditional



$p$ A shape is a rectangle.

$q$ A shape has four sides.

$p \Leftrightarrow q$ A shape is a rectangle if and only if it has four sides.

# WFF

The set of sentences or *well-formed propositional formulae* (WFF) is defined as:

- Any propositional symbol is in WFF.
- The nullary connectives, **true** and **false** are in WFF.
- If A and B are in WFF then so is ~A, A v B, A ^ B, A → B and A ↔ B.
- If A is in WFF then so is (A).

# Example



The following are well formed formulae

$$p \qquad (p) \qquad p \vee q \qquad (((p \Rightarrow q) \wedge \neg q) \Rightarrow \neg p)$$

whereas

are not. $(()) \qquad (p \wedge q) \vee \qquad (p\neg \Rightarrow q)$

# Recap: Truth Tables

| $P$ | $Q$ | $\neg P$ | $P \wedge Q$ | $P \vee Q$ | $P \Rightarrow Q$ | $P \Leftrightarrow Q$ |
|---|---|---|---|---|---|---|
| false | false | true | false | false | true | true |
| false | true | true | false | true | true | false |
| true | false | false | false | true | false | false |
| true | true | false | true | true | true | true |

# Truth Tables for Compound Propositions



- Truth tables may be used to show **interpretations** of compound propositions.
- To draw up a truth table, construct a column for each proposition involved.
- You need $2^n$ rows for n propositions for all possible ways of setting the propositions to T and F.
- If we have 3 propositions, p; q; r, i.e. we need $2^3 = 8$ rows.
- Next, construct a column for each connective, the most deeply nested first.
- Evaluate each column using values for propositions or previous columns.

# Exercise

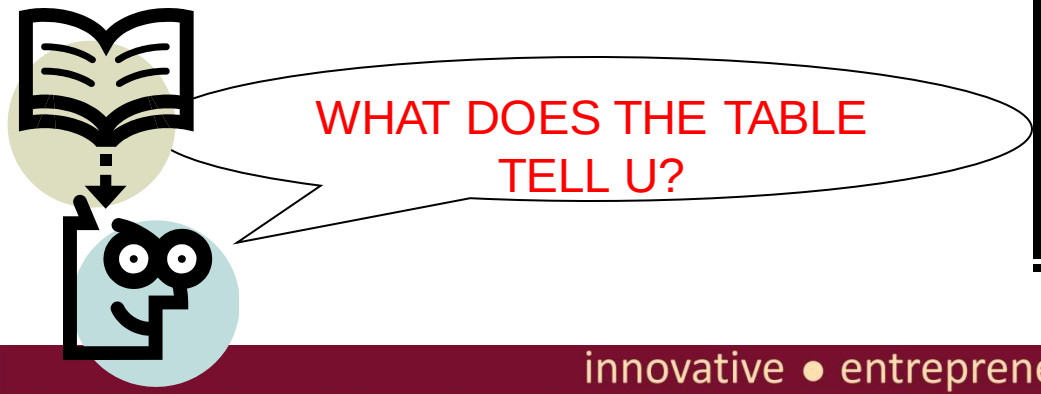
$p$.. (my breakfast is) eggs.

$q$.. (my breakfast is) cereal.

$r$.. (my breakfast is) toast.

•The statement 'my breakfast is either eggs or cereal, and toast' may be written in symbolic form as ???

| $p$ | $q$ | $r$ | $p \vee q$ | $(p \vee q) \wedge r$ |
|---|---|---|---|---|
| $T$ | $T$ | $T$ | | |
| $T$ | $T$ | $F$ | | |
| $T$ | $F$ | $T$ | | |
| $T$ | $F$ | $F$ | | |
| $F$ | $T$ | $T$ | | |
| $F$ | $T$ | $F$ | | |
| $F$ | $F$ | $T$ | | |
| $F$ | $F$ | $F$ | | |



# ... So

The truth table gives results for all possible interpretations of $p$, $q$ and $r$. The compound proposition is true if

- I eat cereal, eggs and toast; or
- I eat cereal and toast; or
- I eat eggs and toast.

*Interpretation - A line in the truth table.*

# Try this..

- Complete the truth table for $\neg((p \Rightarrow q) \vee (q \Rightarrow p))$

| $p$ | $q$ | $(p \Rightarrow q) \vee (q \Rightarrow p)$ | $\neg((p \Rightarrow q) \vee (q \Rightarrow p))$ |
|---|---|---|---|
| $T$ | $T$ | | |
| $T$ | $F$ | | |
| $F$ | $T$ | | |
| $F$ | $F$ | | |

- Show that the formula $(p \wedge q) \Rightarrow r$ satisfiable
- Find an interpretation which satisfies

$$(p \wedge q) \vee (\neg r \wedge s).$$

- Show $p \vee \neg p$ is valid.



# 2.3 Predicate Logic

# Predicate Logic

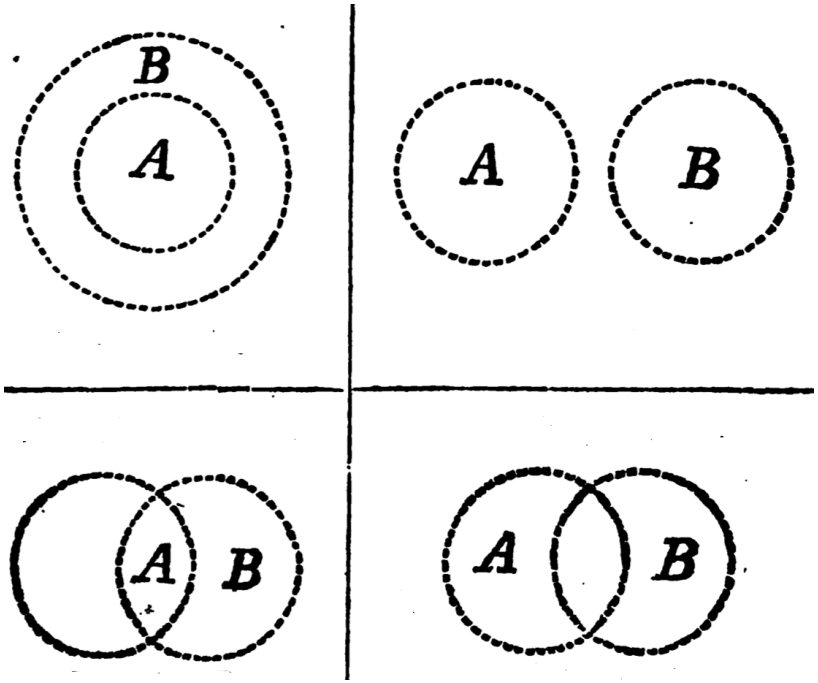
- Propositional logic combines atoms
  - An atom contains no propositional connectives
  - Have no structure (today_is_wet, john_likes_apples)
- **Predicates** allow us to talk about objects
  - Properties: is_wet(today)
  - Relations: likes(john, apples)
  - True or false
- In predicate logic each atom is a predicate
  - e.g. first order logic, higher-order logic

# Activity 2.3

**Activity**

Form a group of two members. Discuss the right logic operation that is suitable for Venn diagram below.

Operation : .....




Operation : .....

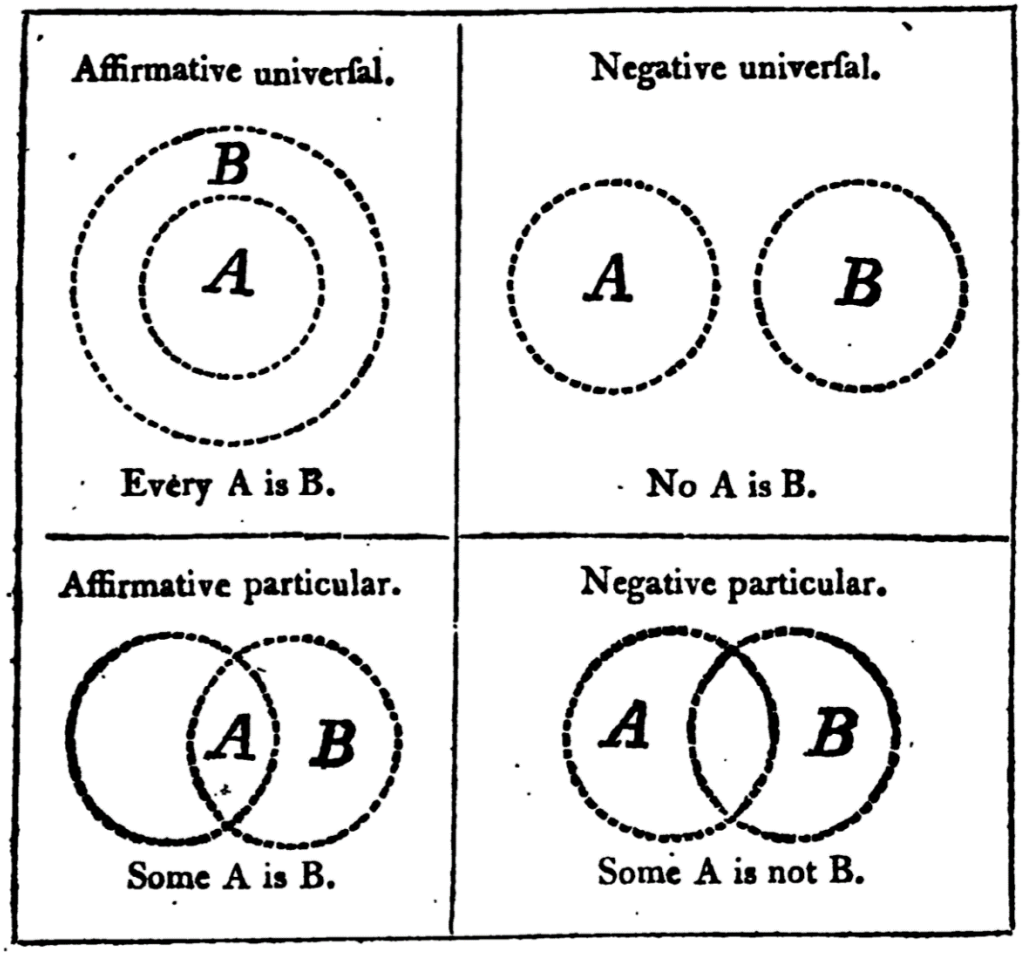
Operation : .....

Operation : .....

# Activity 2.3 - Answer



*Emblems of the four Species of Propositions.*



# First Order Logic

- **Constants** are objects: john, apples
- **Predicates** are properties and relations:
  - likes(john, apples)
- **Functions** transform objects:
  - likes(john, fruit of(apple_tree))
- **Variables** represent any object: likes(X, apples)
- **Quantifiers** qualify values of variables
  - True for every objects (Universal): $\forall X.$ likes(X, apples)
  - Exists at least one object (Existential): $\exists X.$ likes(X, apples)

# Example: FOL Sentence



- "Every rose has a thorn"

$$\forall X.(rose(X) \rightarrow \exists Y.(has(X,Y) \wedge thorn(Y)))$$

- For all X
  - if (X is a rose)
  - then there exists Y
    - (X has Y) and (Y is a thorn)

# Example: FOL Sentence



- "On Mondays and Wednesdays I go to John's house for dinner"

$$\forall X.((is_mon(X) \vee is_wed(X)) \rightarrow eat_meal(me, houseOf(john), X))$$

- Note the change from "and" to "or"
  - Translating is problematic

# First-Order Logic (FOL)



- FOL is a formal language for describing knowledge about objects, their properties and relations
- Constants and variables are associated with objects, while predicates describe relations between objects
- Unlike propositional logic, it can handle infinite domains through the use of quantifiers
- Generalized modus ponens is a sound inference method, but it is complete only for a subset of FOL (clauses with exactly one positive literal)
- Inference in FOL is only semi-decidable: can find a proof if one exists, but if a proof does not exist, the procedure may not halt.

# First-Order Logic (FOL)



- A key element of FOL are *predicates*, which are used to describe objects, properties, and relationships between objects
  E.g. $On(x,y)$
- A *quantified statement* is a statement that applies to a class of objects
  E.g. $\forall x\ On(x, Table) \rightarrow Fruit(x)$
  - This means that there is only fruit on the table
  - The first element is called a *quantifier*, $x$ is a *variable* and *Table* is a *constant*
  - *On* is a *predicate*
- The use of *quantifiers* allows FOL to handle *infinite domains*, while propositional logic can only handle finite domains.

# First Order Predicate Calculus



- Includes 2 symbols:
  - Variable quantifiers
    - ∃ (existential) and
    - ∀ (universal)
- A quantifier followed by a variable and a sentence:
  - ∀ X likes(X,pizza) ;true for all X
  - ∃ Y friends(Y,amir) ;true if there is at least one

# Universal Quantification



- Syntax: $\forall$ *variables sentence*
- E.g. Everyone taking AI is smart.

  $\forall x\ Taking(x,AI) \rightarrow Smart(x)$
- Semantics: $\forall x\ S$ is equivalent to the *conjunction of instantiations* of $S$:

  $Taking(John,AI) \rightarrow Smart(John)$

  $\wedge\quad Taking(Ann,AI) \rightarrow Smart(Ann)$

  $\wedge\quad \ldots$
- Typically, $\rightarrow$ is the main connective with $\forall$.

# Example

- What does this statement mean:

$\forall x\ Taking(x,AI) \wedge Smart(x)$

# Example

- What does this statement mean:

  $\forall x\ Taking(x,AI) \wedge Smart(x)$

- Common mistake: using $\wedge$ as the main connective with $\forall$:

  $\forall x\ Taking(x,AI) \wedge Smart(x)$

  means "Everyone is taking AI and everyone is smart"

# Existential Quantification



- Syntax: $\exists$ *variables sentence*
- Someone taking AI is smart:

  $\exists x\ Taking(x,AI) \wedge Smart(x)$
- Semantics: $\exists x\ S$ is equivalent to the *disjunction of instantiations* of $S$

  $(Taking(Ann,AI) \wedge Smart(Ann))$

  $\vee\quad (Taking(John,AI) \wedge Smart(John))$

  $\vee\quad \ldots$
- Typically, $\wedge$ is the main connective with $\exists$.

# Example

- What does this mean:

$$\exists x \; Taking(x, AI) \rightarrow Smart(x)$$

# Example

- What does this mean:

  $\exists x$ *Taking(x,AI)* $\longrightarrow$ *Smart(x)*

- Common mistake: using $\longrightarrow$ as the main connective with $\exists$:

  $\exists x$ *Taking(x,AI)* $\longrightarrow$ *Smart(x)* is true if there is anyone who is not taking AI!

# Properties of Quantifiers



- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ *is not the same as* $\forall y \exists x$

$\exists x \forall y Loves(x, y)$

$\forall y \exists x Loves(x, y)$

# Properties of Quantifiers



- $\forall x \forall y$ is the same as $\forall y \forall x$
- $\exists x \exists y$ is the same as $\exists y \exists x$
- $\exists x \forall y$ *is not the same as* $\forall y \exists x$

  $\exists x \forall y Loves(x, y)$

  "There is a person who loves everyone in the world"

  $\forall y \exists x Loves(x, y)$

  "Everyone in the world is loved by at least one person"

# Quantifier Duality

Each quantifier can be expressed using the other quantifier and negation:

- $\forall x\ Likes(x, IceCream)$ is equivalent to $\neg\exists x\ \neg Likes(x, IceCream)$
- $\exists x\ Likes(x, Broccoli)$ is equivalent to $\neg\forall x\ \neg Likes(x, Broccoli)$

Everyone likes ice cream == Not someone not liking ice cream
(nobody do not like ice cream)

Someone like broccoli == Not everyone not liking broccoli

# How to read quantified formulas



- When reading quantified formulas in English, **read them from left to right.** $\forall x$ can be read as "for every object $x$ in the universe the following holds" and $\exists x$ can be read as "there exists an object $x$ in the universe which satisfies the following" or "for some object $x$ in the universe the following holds".
- For example, let the universe be the set of airplanes and let $F(x, y)$ denote "$x$ flies faster than $y$". Then
- $\forall x \; \forall y \; F(x, y)$ can be translated initially as "For every airplane $x$ the following holds: $x$ is faster than every (any) airplane $y$". In simpler English it means "Every airplane is faster than every airplane (including itself !)".
- $\forall x \; \exists y \; F(x, y)$ can be read initially as "For every airplane $x$ the following holds: for some airplane $y$, $x$ is faster than $y$". In simpler English it means "Every airplane is faster than some airplane".
- $\exists x \; \forall y \; F(x, y)$ represents "There exist an airplane $x$ which satisfies the following: (or such that) for every airplane $y$, $x$ is faster than $y$". In simpler English it says "There is an airplane which is faster than every airplane" or "Some airplane is faster than every airplane".
- $\exists x \; \exists y \; F(x, y)$ reads "For some airplane $x$ there exists an airplane $y$ such that $x$ is faster than $y$", which means "Some airplane is faster than some airplane".

# Fun with Sentences



- Brothers are siblings

$$\forall x \forall y Brother(x, y) \rightarrow Sibling(x, y)$$

- "Sibling" is reflexive

$$\forall x \forall y Sibling(x, y) \leftrightarrow Sibling(y, x)$$

- One's mother is one's female parent

$$\forall x \forall y Mother(x, y) \leftrightarrow (Female(x) \wedge Parent(x, y))$$

- A first cousin is a child of a parent's sibling

$$\forall x \forall y FirstCousin(x, y) \leftrightarrow \exists p \exists ps Parent(p, x) \wedge Sibling(ps, p) \wedge Parent(ps, y)$$

# Proof methods

Proof methods divide into (roughly) two kinds:

- *Model checking*:
  - Truth table enumeration (sound and complete for propositional logic)
  - Heuristic search in model space (sound but incomplete)
- *Application of inference rules*:
  - Legitimate (sound) generation of new sentences from old
  - A *proof* is a sequence of inference rule applications
  - Inference rules can be used as operators in a standard search algorithm!

# Proofs



The proof process can be viewed as a *search* in which the operators are *inference rules*:

- *Modus Ponens (MP)*

$$\frac{\alpha, \quad \alpha \rightarrow \beta}{\beta} \qquad \frac{Takes(Joe,AI) \quad Takes(Joe,AI) \rightarrow Cool(Joe)}{Cool(Joe)}$$

- *And-Introduction (AI)*

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta} \qquad \frac{Cool(Joe) \quad CSMajor(Joe)}{Cool(Joe) \wedge CSMajor(Joe)}$$

- *Universal Elimination (UE)*

$$\frac{\forall x \alpha}{\alpha\{x/\tau\}} \qquad \frac{\forall x \; Takes(x,AI) \rightarrow Cool(x)}{Takes(Pat,AI) \rightarrow Cool(Pat)}$$

$\tau$ must be a ground term: a term with no variables

# Example Proof

| | |
|---|---|
| Bob is a buffalo | 1. Buffalo(Bob) |
| Pat is a pig | 2. Pig(Pat) |
| Buffaloes outrun pigs | 3. $\forall x \forall y$ Buffalo(x) $\wedge$ Pig(y) $\rightarrow$ Faster(x,y) |
| AI 1 & 2 | 4. Buffalo(Bob) $\wedge$ Pig(Pat) |
| UE 3, x/Bob, y/Pat | 5. Buffalo(Bob) $\wedge$ Pig(Pat) $\rightarrow$ Faster(Bob,Pat) |
| MP 4 & 5 | 6. Faster(Bob,Pat) |

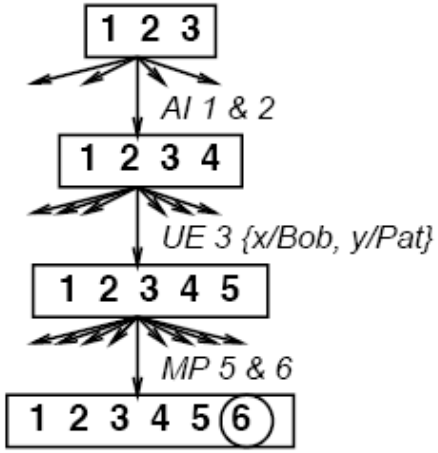
# Search with Primitive Inference Rules



Operators are inference rules

States are sets of sentences

Goal test checks state to see if it contains query sentence




AI, UE, MP is a common inference pattern

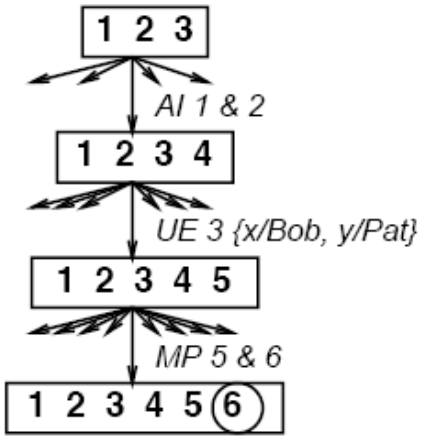
# Search with Primitive Inference Rules



Operators are inference rules

States are sets of sentences

Goal test checks state to see if it contains query sentence




AI, UE, MP is a common inference pattern

Problem: branching factor huge, especially for UE

Idea: find a substitution that makes the rule premise match some known facts

$\Rightarrow$ a single, more powerful inference rule

# Unification

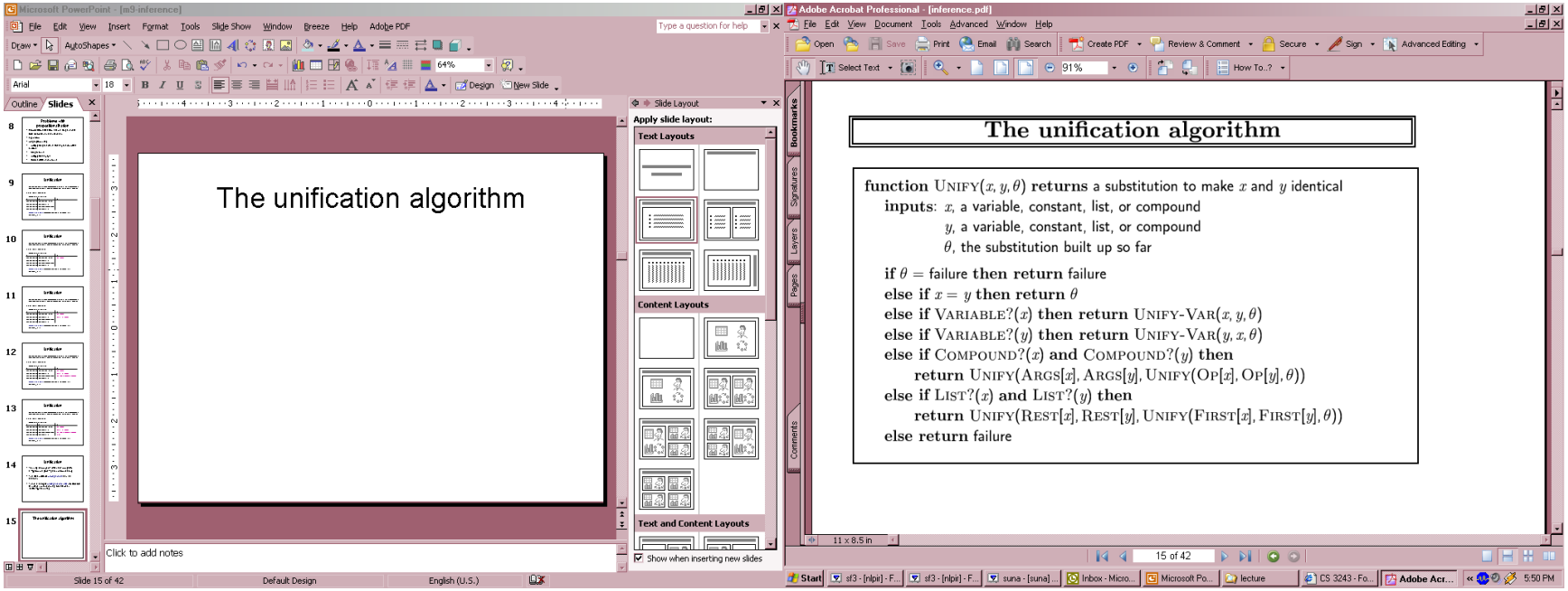
- A *substitution* $\sigma$ unifies atomic sentences $p$ and $q$ if $p\sigma = q\sigma$

| $p$ | $q$ | $\sigma$ |
|---|---|---|
| *Knows(John,x)* | *Knows(John,Jane)* | *x/Jane* |
| *Knows(John,x)* | *Knows(y,Mary)* | *y/John,x/Mary* |
| *Knows(John,x)* | *Knows(y,Mother(y))* | *y/John,x/Mother(John)* |

- Idea: Unify rule premises with known facts, apply unifier to conclusion
- E.g., if we know *q* and the rule: *Knows(John,x)* $\rightarrow$ *Likes(John,x)*, we conclude:
  - *Likes(John,Jane)*
  - *Likes(John,Mary)*
  - *Likes(John,Mother(John))*

# The unification algorithm





# The unification algorithm



**function** UNIFY-VAR($var, x, \theta$) **returns** a substitution

**inputs**: $var$, a variable

$x$, any expression

$\theta$, the substitution built up so far

**if** $\{var/val\} \in \theta$ **then return** UNIFY($val, x, \theta$)
**else if** $\{x/val\} \in \theta$ **then return** UNIFY($var, val, \theta$)
**else if** OCCUR-CHECK?($var, x$) **then return** failure
**else return** add $\{var/x\}$ to $\theta$

# Propositional Resolution



- Resolution rule:
- Resolution refutation:
  - Convert all sentences to CNF
  - Negate the desired conclusion (converted
  - Apply resolution rule until either
    - Derive false (a contradiction)
    - Can't apply any more

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Resolution refutation is sound and complete
  - If we derive a contradiction, then the conclusion follows from the axioms
  - If we can't apply any more, then the conclusion cannot be proved from the axioms.

# Resolution



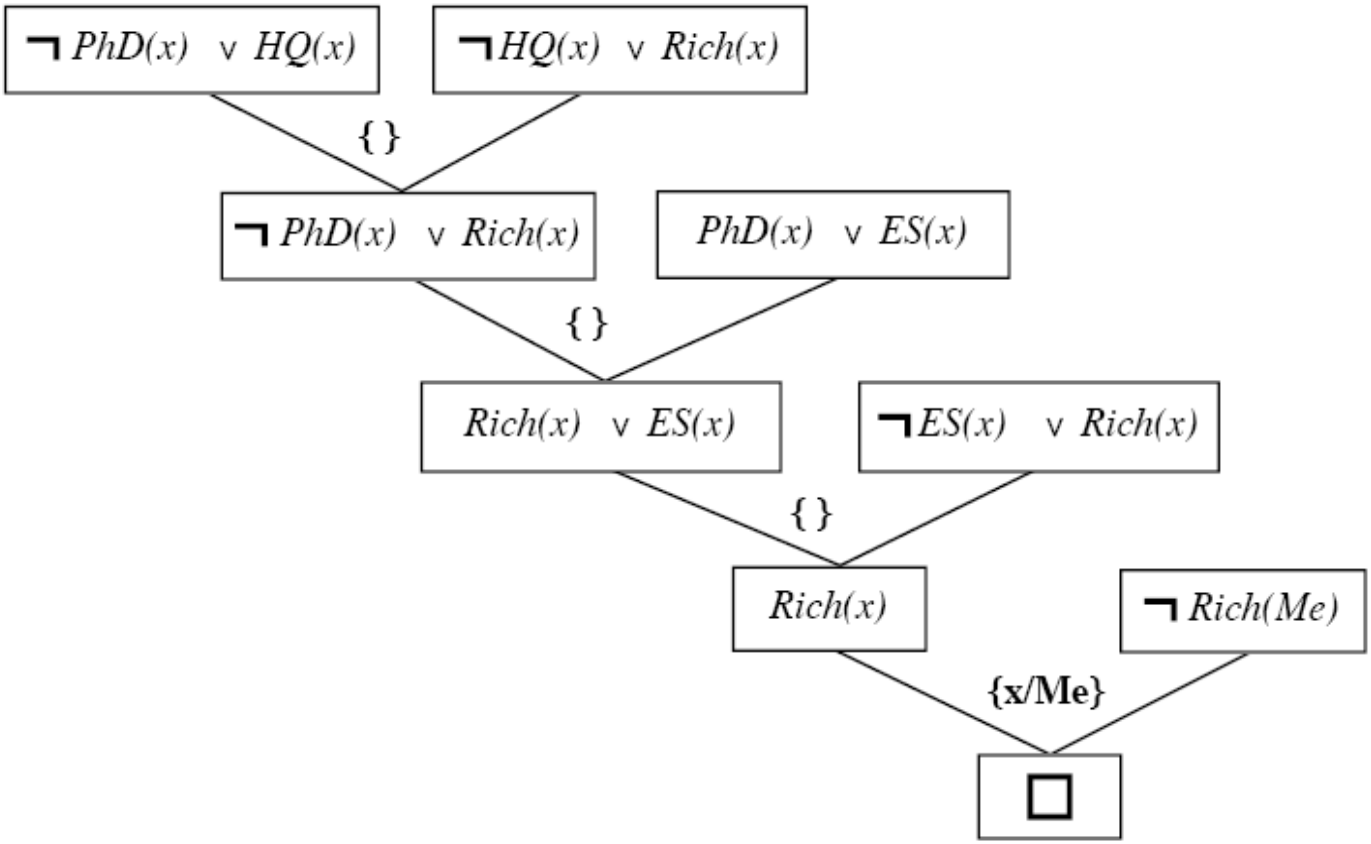
- Resolution is a sound and complete inference method for first-order logic.
- Resolution is a *refutation* procedure: to prove that $KB \models \alpha$, we show that $KB \wedge \neg\alpha$ is unsatisfiable
- The knowledge base and $\neg\alpha$ are expressed in universally quantified, conjunctive normal form
- Like in propositional logic, the resolution inference rule combines two clauses to make a new one:




- Inference continues until an empty clause is derived (contradiction)

# Resolution Proof Tree





# Resolution Strategies



Heuristics that impose a sensible order on the resolutions we attempt:

- *Unit resolution*: prefer to perform resolution if one clause is just a literal - yields shorter sentences
- *Set of support*: identify a subset of the *KB* (hopefully small); every resolution will take a clause from the set and resolve it with another sentence, then add the result to the set of support
  - *Can make inference incomplete!*
- *Input resolution*: always combine a sentence from the query or *KB* with another sentence
  - Modus ponens is a kind of input resolution
  - Not compete in general

# In resolution refutation, can we use a clause that have been resolved?

- In resolution if we have a set S composed of three clause C1, C2 and C3 and we want to proof that C4 is derivable from S using refutation:
- Suppose we've resolved C1 and C2 to C5, can we resolve C1 with C5 or C3 or C4 or C2 cause even it has been resolved?
- And if we want to conceive an algorithm for that do we have to delete each resolved couple of clauses from or initial set I= {S, ¬C4} and add the resolvent instead?
- And lastly, do we have to use all the clause to derive ⊥ or we stop once we find ⊥ even if we find it in the first step?

# Answers

- Suppose we've resolved C1 and C2 to C5, can we resolve C1 with C5 or C2?

  No, not really. If C1 and C2 can be resolved in more than one way, then all those resolvents are tautologies. And if C1 can be resolved with C5, then it can also be resolved with C2 in a second way (assuming C1 is not a tautology, and hence cannot be resolved with itself).

# Answers..

- Can we resolve C1 with C3 or C4 cause even it has been resolved?

  We can (and often must) resolve C1 with C3, even so it has already been resolved with C2. We don't need to resolve any clause with C4 itself, rather the negation of C4 must be brought into a conjunctive normal form. The clauses of that normal for have to be added to our set S.

# Answers ..

- And if we want to conceive an algorithm for that do we have to delete each resolved couple of clauses from or initial set I= {S, ¬C4} and add the resolvent instead?
  - No, that would be too simple. What you actually do is to select a variable that you want to eliminate from the clauses in the set S, and then:
  - for every clause C containing the variable and every clause N containing the negation of the variable
  - resolve C and N and add the resolvent to the set S
  - remove all original clauses containing the variable or its negation from the set S

# Answers..

- And lastly, do we have to use all the clause to derive $\perp$ or we stop once we find $\perp$ even if we find it in the first step?

  We stop once we find $\perp$.

# Example: Translate the KB into propositional logic



- If it is hot and humid, then it is raining. If it is humid, then it is hot. It is humid.

| | |
|---|---|
| H | It is hot. |
| D | It is humid. |
| R | It is raining. |

1. If it is hot and humid, then it is raining
2. If it is humid, then it is hot
3. It is humid

# Example: PROOF-Logical Inference Rules



GOAL-Is it Raining?

1. (H ^ D) → R
2. D → H
3. D

From 2 and 3: by Modus Ponens, we infer:

4. H

From 4: by ^-introduction, we infer:

5. H ^ D

From 1 and 5: by Modus Ponens, we infer:

6. **R** (Goal -- It is raining)

# Applications of First-Order Logic



- Prolog: a logic programming languages
- Production systems
- Semantic nets
- Automated theorem proving
- Planning

# Summary

1. We learned three topics in this chapter: AI knowledge representation, propositional logic and predicate logic.
2. Knowledge representation is ta part of Artificial intelligence which concerned with thinking that contributes to intelligent behavior.
3. Propositional logic is a branch of mathematical logic which studies the logical relationships between propositions (or statements, sentences, assertions) taken as a whole, and connected via logical connectives.
4. A predicate is an expression of one or more variables defined on some specific domain. A predicate with variables can be made a proposition by either assigning a value to the variable or by quantifying the variable